



Le génie pour l'industrie

GUIDE PROGRAMMEUR

PRÉSENTÉ À L'ÉCOLE DE TECHNOLOGIE SUPÉRIEURE
DANS LE CADRE DU COURS
LOG795 PROJET DE FIN D'ÉTUDES EN GÉNIE LOGICIEL

DÉVELOPPEMENT D'UN OUTIL D'ÉTIQUETAGE AUTOMATIQUE DES PULL REQUESTS
VIA LES GRANDS MODÈLES DE LANGAGE (LLMS)

FAROUK MEBARKI MEBF71040201
TALIP KOYLUOGLU KOYT85110004
CHAWKI BENZEGHIBA BENC63050101
MEHDI YASSIN YASM24019802
MIA BEN REDJEB BENM69270201

DÉPARTEMENT DE GÉNIE LOGICIEL ET DES TI

Professeur-superviseur

ALI OUNI

MONTREAL, 10 AOÛT 2026
ÉTÉ 2026

Table des matières

1. Objet du guide et limites du système	3
2. Organisation du dépôt et architecture.....	4
3. Environnement local et démarrage	5
4. Modèle d'exécution et événements GitHub.....	6
5. Lecture des données et écritures GitHub	8
6. Sélection et classement des fichiers	9
7. Budget de jetons et construction du prompt	10
8. Architecture des fournisseurs LLM	11
9. Configuration, OAuth et persistance	12
10. Modèle de sécurité	13
11. Commentaires, labels et Check Runs	14
12. Tests, évaluation et diagnostic	15
13. Extension, déploiement et passation.....	16

1. Objet du guide et limites du système

1.1 Public cible

Ce document s'adresse au développeur qui reprend PRIntentLabeler et doit pouvoir compiler le projet, reproduire un environnement local, comprendre les décisions architecturales et modifier un flux sans affaiblir les protections existantes. Il suppose une maîtrise de TypeScript, Node.js, HTTP, PostgreSQL, GitHub Apps et des principes de test automatisé.

1.2 Responsabilités de PRIntentLabeler

- Recevoir les webhooks GitHub liés aux Pull Requests, aux commentaires, aux Check Runs et aux installations.
- Lire les métadonnées, les fichiers et les labels du dépôt avec Octokit, puis construire un contexte LLM borné.
- Obtenir au plus trois suggestions de labels existants, valider la réponse et publier un commentaire idempotent.
- Appliquer uniquement des variantes de labels identifiables comme provenant de l'IA.
- Gérer une configuration LLM distincte par installation GitHub, avec OAuth, chiffrement et audit.

1.3 Limites assumées

L'application ne révise pas la qualité du code, ne décide pas si une PR peut être fusionnée et ne crée pas une taxonomie libre. Le LLM doit choisir dans le catalogue du dépôt. Le traitement est réalisé dans le processus du webhook : il n'existe actuellement ni file de messages, ni worker asynchrone, ni cache distribué. Une panne du fournisseur ne doit toutefois pas empêcher la publication d'un commentaire explicatif.

1.4 Invariants à préserver

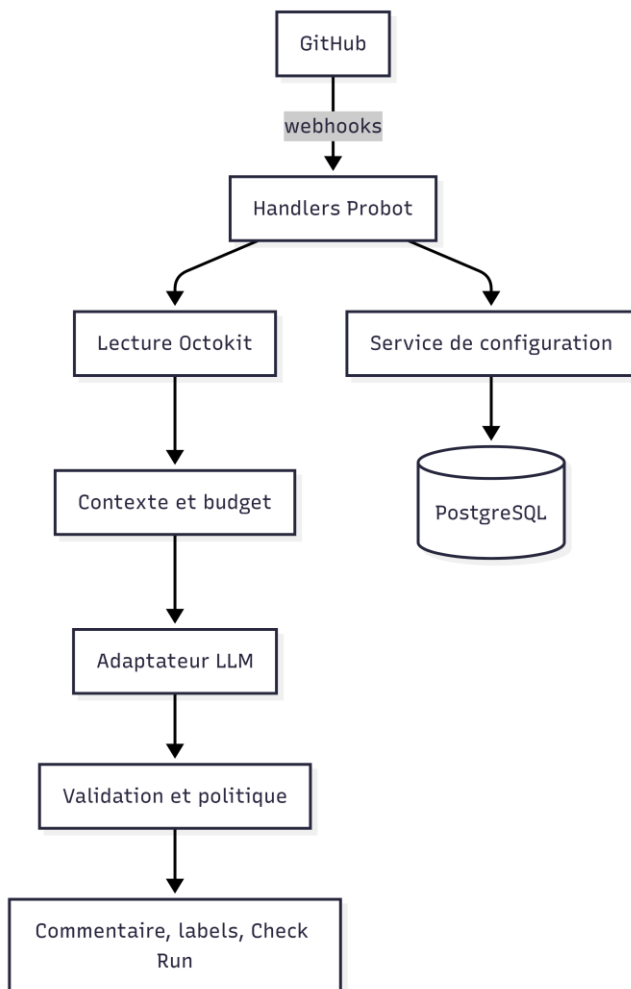
Invariant	Conséquence technique
Données GitHub non fiables	Échapper, borner et délimiter tout texte injecté dans le prompt ou le HTML.
Action sur analyse courante	Comparer le SHA stocké, le SHA de la Check Run et le SHA actuel de la PR.
Aucun label inventé	Valider le nom contre le catalogue GitHub, sans tenir compte de la casse.
Label humain intact	Créer et gérer uniquement la variante préfixée par l'icône de robot.
Secret non réaffichable	Conserver la clé chiffrée et seulement ses quatre derniers caractères en clair.

2. Organisation du dépôt et architecture

2.1 Découpage par responsabilité

Répertoire	Responsabilité
src/handlers	Orchestration des événements; aucune logique cryptographique ou SQL directe.
src/github	Lecture et écriture GitHub, pagination, commentaire idempotent et Check Run.
src/llm	Sélection, budget, prompt, adaptateurs, validation de sortie et reprise sur erreur.
src/labels	Politique de labels, modes d'application et synchronisation des cases.
src/configuration	Cas d'utilisation de configuration et abstraction de persistance.
src/auth, src/security	OAuth, sessions, CSRF, chiffrement et état signé.
src/http	Routes Express et rendu HTML de l'interface de configuration.
scripts, evaluation, test	Migration, diagnostic réel, évaluation reproductible et tests Vitest.

Schéma Mermaid - vue logique du traitement



2.2 Points d'entrée

src/server.ts lance Probot avec run(app). src/index.ts enregistre les routes HTTP et les abonnements. Les handlers constituent la frontière applicative; ils composent des fonctions testables et journalisent le contexte owner/repo/pullNumber. Les types de src/domain transportent les données entre les couches sans dépendre d'Express ou de PostgreSQL.

Carte de navigation minimale

```
src/index.ts
pull_request.*      -> handlePullRequestEvent
check_run.requested_action -> handleCheckRunRequestedAction
issue_comment.edited -> handleIssueCommentEdited
installation.deleted -> configurationService.delete
HTTP /setup, /settings -> registerConfigurationRoutes
```

Règle de dépendance : Une nouvelle règle de domaine doit rester dans llm, labels, commentaires ou configuration. Le handler orchestre; il ne doit pas devenir l'endroit où se concentrent les règles métier.

3. Environnement local et démarrage

3.1 Prérequis

Le dépôt exige Node.js 20 ou plus, npm, Git et Docker Desktop. PostgreSQL 17 est fourni par compose.yml. Un compte GitHub capable de créer une GitHub App, un canal Sme.io et au moins une clé de fournisseur LLM sont nécessaires pour un essai bout en bout.

Installation reproductible

```
git clone https://github.com/stilab-ets/PRIntentLabeler.git
cd PRIntentLabeler
npm ci
docker compose up -d postgres
npm run db:migrate
npm run dev
```

3.2 Variables d'environnement

Variable	Rôle	Obligation
APP_ID, PRIVATE_KEY	Authentification de la GitHub App	Oui
WEBHOOK_SECRET, COMMENT_STATE_SECRET	Webhook et signature HMAC du commentaire	Oui / facultatif
WEBHOOK_PROXY_URL	Canal Sme.io pour le développement local	Local
DATABASE_URL, DATABASE_SSL, CONFIG_ENCRYPTION_KEY	PostgreSQL et clé AES de 32 octets	Configuration UI
GITHUB_CLIENT_ID, GITHUB_CLIENT_SECRET, PUBLIC_BASE_URL	OAuth utilisateur et callback /settings	Oui pour OAuth
GROQ_API_KEY, GROQ_MODEL	Fournisseur de repli du serveur	Facultatif
LABEL_MODE	suggest, auto-high ou auto-all	Défaut suggest

Commandes de verification

```
# Génération de la clé maîtresse
openssl rand -base64 32

# Contrôles initiaux
docker compose ps
npm run build
npm test
```

3.3 Paramètres GitHub App

Configurer Setup URL sur `http://localhost:3000/setup`, Callback URL sur `http://localhost:3000/auth/github/callback` et faire correspondre `PUBLIC_BASE_URL`. Les événements requis sont `pull_request`, `check_run`, `issue_comment` et `installation`. Les permissions sont `metadata:read`, `pull_requests:read`, `issues:write` et `checks:write`. Une modification de `app.yml` ne met pas automatiquement à jour une application déjà créée dans GitHub.

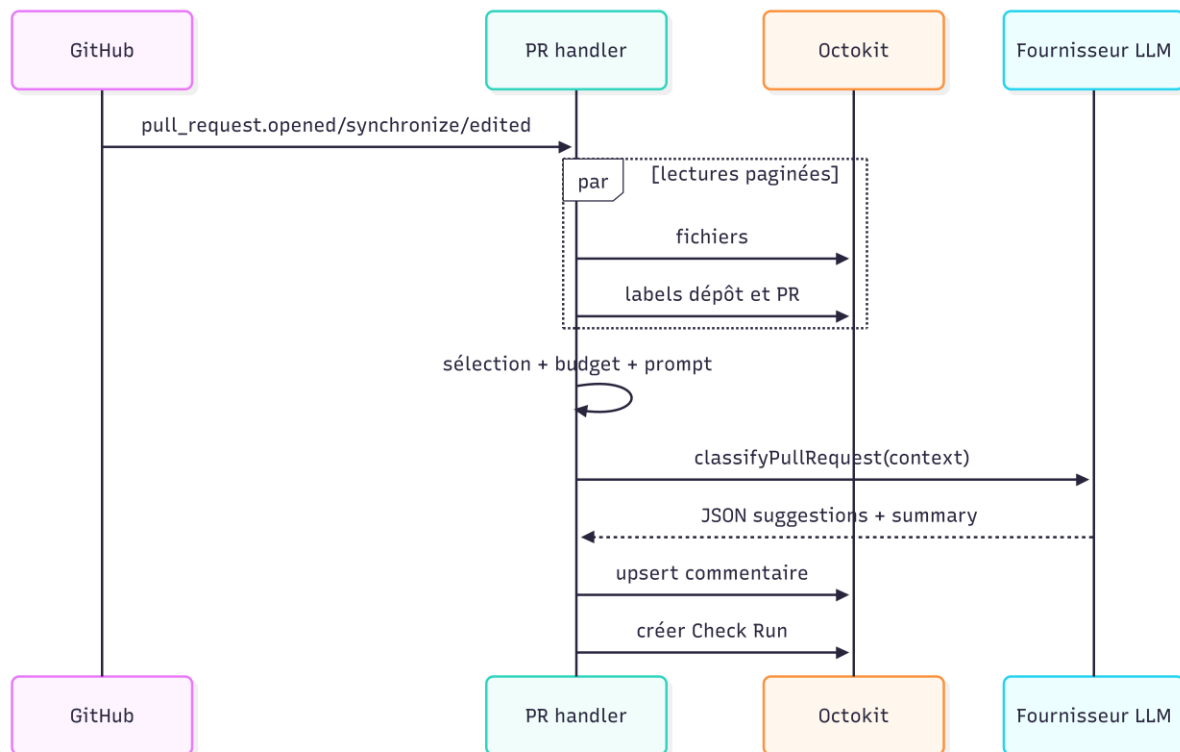
Démarrage réussi : Le terminal doit indiquer le chargement de l'application, la connexion au relais Smee et l'écoute sur le port 3000. Ouvrir ensuite `/settings`, enregistrer un fournisseur, puis créer une PR de tes

4. Modèle d'exécution et événements GitHub

4.1 Abonnements

Événement	But	Effet
<code>pull_request.opened</code>	Première analyse	Lit la PR, appelle le LLM et publie les sorties.
<code>pull_request.synchronize</code>	Nouveau commit	Reconstruit l'analyse pour le nouveau head SHA.
<code>pull_request.edited</code>	Titre ou description	Relance la classification avec le nouveau contexte.
<code>check_run.requested_action</code>	Bouton GitHub	Réutilise l'état signé sans rappeler le LLM.
<code>issue_comment.edited</code>	Case cochée	Synchronise seulement les labels suggérés par le bot.
<code>installation.deleted</code>	Désinstallation	Supprime la configuration de l'installation.

Schéma Mermaid - séquence principale d'analyse



4.2 Frontières d'échec

`handlePullRequestEvent` contient un `try/catch` global afin qu'une exception ne fasse pas tomber le processus Probot. L'appel LLM possède sa propre frontière : une erreur devient `llmFailureReason` et le commentaire est tout de même produit. La création de la Check Run est également non bloquante, car la permission `checks:write` peut manquer. L'application de labels échoue en mode dégradé et conserve les suggestions.

Principe de dégradation contrôlée

```

try {
  const raw = await provider.classifyPullRequest(llmContext);
  analysis = { ...raw, suggestions: filterValidSuggestions(...) };
} catch (error) {
  llmFailureReason = describeLlmFailure(error);
}
await upsertPullRequestComment(context, number, commentBody);

```

Contrainte actuelle : Le traitement demeure synchrone dans le webhook. Une évolution vers une file de travaux devra conserver l'idempotence, le head SHA et l'identité de l'installation dans le message.

5. Lecture des données et écritures GitHub

5.1 Construction de PullRequestData

readPullRequestData est utilisé lorsque le payload contient la PR. fetchPullRequestData reconstruit la même structure depuis un événement check_run. Les deux récupèrent en parallèle les fichiers, les labels du dépôt et les labels déjà présents. listAllPages centralise la pagination avec per_page=100; ne pas remplacer cette fonction par un seul appel Octokit dans un nouveau flux.

Lecture parallèle et paginée

```
const [rawFiles, rawLabels, rawPrLabels] = await Promise.all([
  listAllPages(octokit, octokit.pulls.listFiles, params),
  listAllPages(octokit, octokit.issues.listLabelsForRepo, params),
  listAllPages(octokit, octokit.issues.listLabelsOnIssue, params),
]);
```

PullRequestData conserve le headSha, les statistiques, les branches, les fichiers avec patch facultatif, le catalogue de labels et leurs descriptions. Les labels déjà préfixés par le robot sont exclus du catalogue envoyé au LLM; ils représentent un état d'application, pas une intention disponible.

5.2 Écritures idempotentes

Le commentaire commence par <!-- llm-pr-labeler -->. findBotComment exige ce marqueur et vérifie que le commentaire appartient réellement à la GitHub App. upsertComment met à jour l'élément existant au lieu d'en créer un à chaque synchronisation. APP_ID renforce la vérification de performed_via_github_app.

Écriture	API Octokit	Protection
Commentaire	issues.createComment/updateComment	Marqueur et propriétaire bot
Label IA	issues.createLabel/addLabels	Préfixe distinct et 422 ignoré si déjà créé
Retrait	issues.removeLabel	Uniquement variante IA suggérée
Check Run	checks.create	Échec non bloquant

5.3 Points d'attention

- Un patch GitHub peut être absent pour un binaire ou un diff trop volumineux; le code doit accepter undefined.
- Les appels de liste sont sensibles aux limites API; conserver la pagination et la parallélisation raisonnable.
- Le commentaire est un stockage visible et modifiable : ne jamais lui faire confiance sans signature et contrôle d'identité.
- Une nouvelle écriture GitHub doit être répétable; un webhook peut être livré plus d'une fois.

Diagnostic Les logs owner, repo, pullNumber et commentId permettent de relier une requête GitHub aux sorties. Ne jamais journaliser un patch complet, une clé API ou un jeton OAuth.

6. Sélection et classement des fichiers

6.1 Rôle exclusif et politique de contenu

classifyFileRole applique des règles ordonnées. Le premier rôle correspondant gagne : un workflow YAML reste ci-cd même s'il ressemble à de la configuration, et un fichier sous tests reste test même si son extension est du code source. Le rôle sémantique est distinct de contentPolicy.

Rôle	Score de base	Exemples
source / database	14 / 13	src/*.ts, migrations, SQL, Prisma
ci-cd / dependency	12 / 12	.github/workflows, Dockerfile, package.json
configuration	10	.env, tsconfig, YAML hors CI
documentation / test	9 / 9	README, docs, *.test.ts, fixtures
other / asset / generated	7 / 5 / 1	fichiers non classés, styles, binaires ou dist

La politique include-patch est réservée à un patch textuel utile. Les lockfiles, snapshots, fichiers générés, binaires et fichiers sans patch restent summary-only. Ils comptent dans le résumé global, mais ne consomment jamais un emplacement de diff.

6.2 Calcul du score

Le score part de la valeur du rôle. S'ajoutent une correspondance avec le titre (+3) ou, à défaut, la description (+1), les signaux security/performance (+1 ou +3 si confirmés par le titre), le statut added (+1), la taille du diff (-1 à +2), la présence d'un patch (+1) et le malus fixture (-2). Le score est une priorité de tri, jamais une note de qualité et jamais une preuve envoyée au modèle.

Exemple de classement

```
PR : fix(auth): validate OAuth state
src/auth/github-oauth-service.ts
  source 14 + titre 3 + sécurité 3 + diff 2 + patch 1 = 23 pts
test/github-oauth-service.test.ts
  test 9 + titre 3 + sécurité 3 + diff 2 + patch 1 = 18 pts
```

6.3 Diversité et sélection

selectRepresentativeFiles applique une sélection gloutonne. À chaque tour, le score ajusté vaut score - 2 x nombre du même rôle - 1 x nombre du même groupe de répertoires + 3 si le rôle est explicitement préféré par le titre. Cette décroissance évite qu'une série de tests ou de fichiers du même module masque le fichier principal.

Modification sûre Toute nouvelle extension, règle de rôle ou pondération doit être ajoutée dans file-selector.ts avec des tests de précédence, de contentPolicy, de stabilité du tri et de diversité.

7. Budget de jetons et construction du prompt

7.1 Calcul du budget

Le fournisseur est créé avant le contexte afin que la fenêtre du modèle gouverne la sélection.

resolveLlmTokenBudget recherche d'abord LLM_CONTEXT_WINDOW_TOKENS, ensuite un préfixe de modèle, puis une valeur par défaut du fournisseur et enfin un repli prudent de 16 384 jetons.

Formules appliquées par model-budget.ts et pr-context.ts

```
W = fenêtre du modèle
R = min(3000, floor(W * 0.25))           # réponse et réflexion
P = max(0, floor(W * 0.85) - R)          # prompt sûr
N = estimation système + contexte sans patches
B = max(0, min(10000, P - N))            # budget total des diffs
```

L'estimation locale utilise trois caractères par jeton par défaut. Chaque patch est limité à 60 lignes et 750 jetons. Un nouveau fichier n'est admis que s'il peut recevoir au moins 120 jetons utiles, sauf le premier qui peut être conservé même très tronqué. allocatePatchTokenBudgets distribue d'abord une part équitable, puis redistribue le reste aux demandes non satisfaites.

Les surcharges opérationnelles sont LLM_CONTEXT_WINDOW_TOKENS, ESTIMATED_CHARS_PER_TOKEN et MAX_TOTAL_PATCH_TOKENS. Toute modification doit être corrélée aux métriques de jetons réels plutôt qu'ajustée sur une seule PR.

7.2 Structure du prompt

Le prompt contient les métadonnées, une description nettoyée et limitée à 1 500 caractères, les statistiques, un résumé de 30 fichiers, les rôles, les diffs représentatifs et au plus 30 labels candidats. Le score interne n'est pas rendu. Les marqueurs BEGIN/END UNTRUSTED provenant du dépôt sont neutralisés afin qu'un contributeur ne puisse pas fermer artificiellement une section.

Contrat de sortie attendu

```
{
  "suggestions": [
    { "name": "exact label", "confidence": 0.92,
      "reason": "Direct factual evidence in at most 15 words" }
  ],
  "summary": "One factual sentence describing the implemented change."
}
```

7.3 Validation en profondeur

classification-parser extrait un objet JSON, rejette une réponse vide ou non analysable et ne conserve que des confiances numériques finies dans [0,1]. filterValidSuggestions effectue une seconde validation contre les labels GitHub, rétablit leur casse canonique, élimine les doublons après tri, applique le seuil de 0,70 et limite le résultat à trois. Ce double contrôle doit rester même si un fournisseur impose un schéma JSON natif.

Mesure Les adaptateurs OpenAI compatibles et Gemini journalisent les jetons réels et l'erreur de l'estimation. Surveiller promptTokenErrorPercentage avant de changer ESTIMATED_CHARS_PER_TOKEN.

8. Architecture des fournisseurs LLM

8.1 Contrat commun et fabrique

Interface de `src/llm/llm-provider.ts`

```
export interface LlmProvider {  
  readonly tokenBudget?: LlmTokenBudget;  
  classifyPullRequest(context: PullRequestLlmContext):  
    Promise<PullRequestAnalysis>;  
  checkConnection?(): Promise<void>;  
}
```

`createLlmProvider` normalise la clé, le modèle et l'URL, résout le budget une seule fois, puis instancie l'adaptateur. `createEnvironmentLlmProvider` fournit uniquement le repli Groq lorsque aucune configuration par installation n'est disponible.

Fournisseur	Protocole	Particularité
Groq, OpenAI, xAI	Chat Completions	JSON mode selon l'adaptateur; max_completion_tokens pour OpenAI.
Anthropic	Messages API	x-api-key et anthropic-version.
Gemini	generateContent	responseSchema JSON et jetons de réflexion comptabilisés.
Perplexity	Compatible OpenAI	Adaptateur dédié avec URL Sonar.
Custom	Compatible OpenAI	URL obligatoire, HTTPS en production.

8.2 Reprise et classification des erreurs

`retryLlmRequest` effectue trois tentatives pour 408, 425, 429, 500, 502, 503, 504 et les erreurs réseau usuelles. Le délai exponentiel commence à 600 ms, ajoute du jitter et respecte Retry-After sans dépasser 4 s. Les erreurs 401, 403 et 404 ne sont pas réessayées. `LlmProviderRequestError`, `LlmEmptyResponseError` et `LlmInvalidResponseError` permettent de produire un message utilisateur distinct.

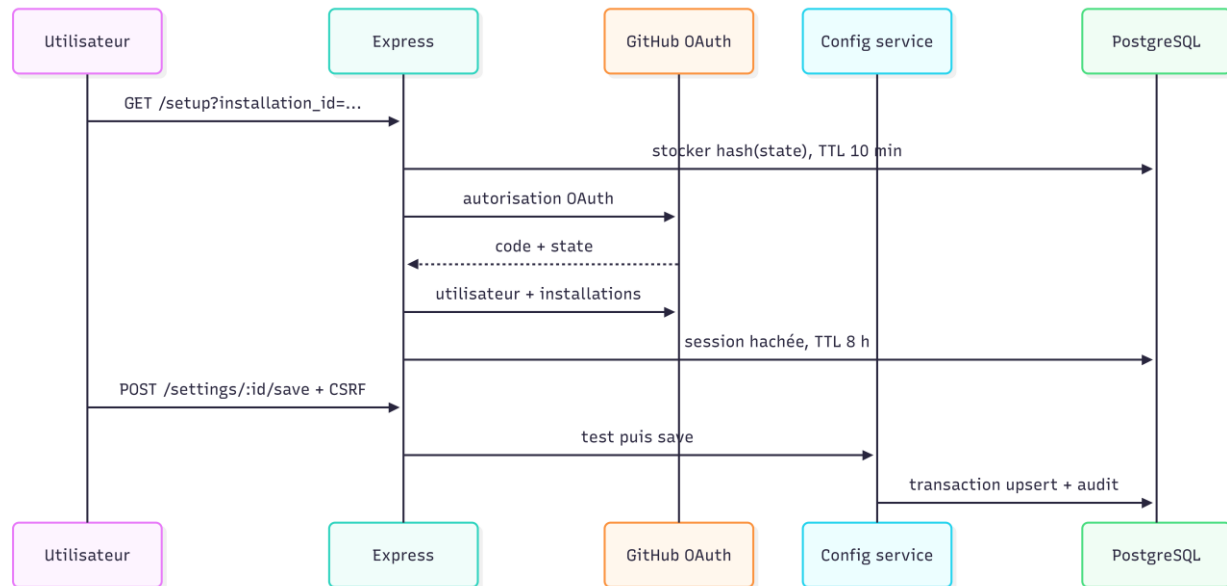
8.3 Ajouter un fournisseur

- Ajouter le nom, l'URL, le modèle par défaut et les suggestions dans `provider-configuration.ts`.
- Créer un adaptateur qui implémente `LlmProvider`, un timeout de 30 s et `checkConnection`.
- Mapper les erreurs HTTP vers `LlmProviderRequestError` et utiliser `retryLlmRequest` pour la classification.
- Déclarer la fenêtre du fournisseur ou les préfixes de modèles dans `model-budget.ts`.
- Brancher l'adaptateur dans `provider-factory.ts` sans exposer la clé aux logs.
- Ajouter des tests d'en-têtes, de payload, de réponse vide, d'erreur et de budget.

9. Configuration, OAuth et persistance

9.1 Parcours de configuration

Schéma Mermaid - OAuth et enregistrement d'une configuration



9.2 Routes et contrôles d'accès

Route	Fonction	Contrôle principal
GET /setup, /auth/github	Démarre OAuth	State aléatoire à usage unique
GET /auth/github/callback	Crée la session	Code, state et installation accessible
GET /settings[/id]	Liste et formulaire	Cookie de session et installation autorisée
POST .../llm/test	Teste la connexion	Session, CSRF, limites de champs
POST .../save/delete	Persiste ou supprime	Session, CSRF, transaction et audit

9.3 Service et schéma

LlmConfigurationService accepte une nouvelle clé ou réutilise la clé chiffrée existante. Un changement de fournisseur exige une nouvelle clé. save teste toujours la connexion avant l'upsert. llm_configurations utilise installation_id comme clé primaire; llm_configuration_audits conserve created, updated ou deleted sans jamais contenir le secret.

Modèle de données simplifié

```
llm_configurations
  installation_id PK | provider | model | encrypted_api_key
  encryption_iv | encryption_auth_tag | key_last_four | base_url
  created_by_github_user_id | updated_by_github_user_id | timestamps

oauth_states(state_hash PK, installation_id, expires_at)
web_sessions(session_hash PK, installation_ids[], csrf_token, expires_at)
```

Migration : Les migrations ne sont pas exécutées au démarrage. `npm run db:migrate` doit faire partie du déploiement. Une nouvelle migration doit être additive, rejouable et testée sur une base contenant déjà `001_llm_configuration.sql`.

10. Modèle de sécurité

10.1 Secrets et chiffrement

ApiKeyCipher exige exactement 32 octets décodés depuis `CONFIG_ENCRYPTION_KEY`. Chaque chiffrement AES-256-GCM utilise un IV aléatoire de 12 octets et produit un ciphertext et une balise d'authentification. La base conserve ces trois valeurs en base64. Un changement non planifié de clé maîtresse rend toutes les clés fournisseurs indéchiffrables; prévoir une procédure de rotation avant toute mise en production durable.

Secret	Stockage autorisé	Interdit
PRIVATE_KEY	Gestionnaire de secrets / variable d'environnement	Dépôt, log, commentaire
Clé fournisseur	Chiffrée en PostgreSQL; suffixe de 4 caractères	HTML, audit, log
Jeton OAuth	Mémoire pendant le callback seulement	Base de données
Session web	Cookie HttpOnly; hash SHA-256 en base	Jeton brut en base

10.2 Défenses applicatives

- OAuth state haché, expirant après 10 minutes et consommé une seule fois; session aléatoire de 8 heures.
- Cookie HttpOnly, SameSite=Lax et Secure en production; CSRF comparé avec timingSafeEqual.
- CSP restrictive, X-Frame-Options DENY, nosniff et Referrer-Policy no-referrer sur les routes HTTP.
- HTML échappé au rendu; longueurs limitées à 160, 4 096 et 2 048 caractères pour modèle, clé et URL.
- URL custom sans identifiants, query ou fragment; HTTPS requis et localhost refusé en production.
- Prompt qui traite le dépôt comme donnée non fiable et neutralise les faux délimiteurs UNTRUSTED.

10.3 État signé et anti-rejeu

renderAnalysisDataBlock encode l'analyse et le headSha en base64. La version 3 est signée par HMAC-SHA256 avec COMMENT_STATE_SECRET ou, à défaut, WEBHOOK_SECRET. Les formats historiques v1 et v2 restent lisibles mais verified=false; les handlers refusent d'agir. Une action exige ensuite l'égalité entre le SHA stocké, celui de la Check Run et le SHA courant de la PR.

Condition de confiance

```
autoriser_action = commentaire_officiel
&& signature_HMAC_valide
&& stored.headSha === checkRun.head_sha
&& stored.headSha === currentPullRequest.headSha
&& labels_demandés ⊆ suggestions_validées
```

11. Commentaires, labels et Check Runs

11.1 Politique de suggestions

Après le parsing, filterValidSuggestions reconstruit des objets propres, impose une confiance minimale de 0,70, trie par confiance, conserve le meilleur doublon, rétablit le nom canonique du dépôt et coupe à trois. LABEL_MODE décide ensuite de l'application initiale.

Mode ou action	Comportement
suggest / Suggest only	Ne pose aucun nouveau label; reflète ceux déjà appliqués dans les cases.
auto-high / Auto-apply high	Applique les suggestions à 0,85 ou plus et retire les variantes suggérées sous ce seuil.
auto-all / Auto-apply all	Applique toutes les suggestions validées, maximum trois.

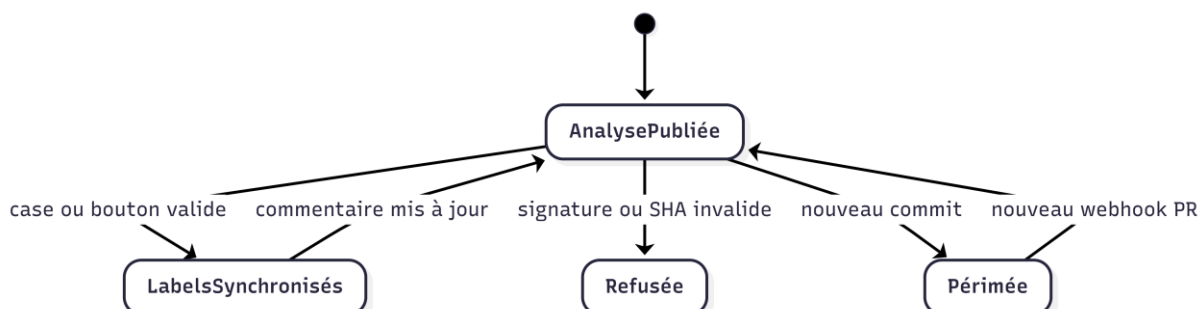
11.2 Variante de label IA

applyAiSuggestedLabels transforme bug en une variante préfixée par l'icône de robot. ensureAiLabelExists tente de lire la couleur du label original, crée la variante et tolère le code 422 lorsque celle-ci existe déjà. Les suppressions ne visent que les noms préfixés; un label humain identique reste intact.

11.3 Synchronisation par commentaire

issue_comment.edited est ignoré si l'auteur est un bot, si la cible n'est pas une PR, si le marqueur manque, si le commentaire n'appartient pas à l'application ou si une case ne porte pas un nom IA. Le handler retrouve ensuite le commentaire officiel, vérifie la signature et le SHA, compare toutes les cases aux suggestions et au catalogue du dépôt, puis calcule toAdd et toRemove dans label-sync.ts.

Schéma Mermaid - cycle de vie de l'analyse interactive



11.4 Check Run

createLabelerCheckRun crée une conclusion neutral et les trois actions permises par GitHub. Le clic émet check_run.requested_action. Le handler relit l'état du commentaire et ne rappelle jamais le LLM; il reconstruit seulement le commentaire après l'action. Une nouvelle action doit obtenir un identifiant stable, une branche explicite dans le handler et des tests de SHA périmé.

12. Tests, évaluation et diagnostic

12.1 Commandes de qualité

Barrière de validation

```
npm run build      # TypeScript strict, sortie lib/
npm run lint       # ESLint sur src, test et scripts
npm run format:check # conformité Prettier
npm test           # Vitest en exécution unique
npm run test:coverage # rapport texte et HTML
npm run test:watch  # boucle locale
```

La configuration Vitest couvre test/**/*.test.ts et la couverture V8 cible src/**/*.ts, sauf src/index.ts. Aucun seuil minimal n'est imposé dans vitest.config.ts; la réussite du rapport de couverture ne prouve donc pas à elle seule une couverture suffisante. Au commit documenté, 32 fichiers et 257 tests réussissent.

12.2 Stratégie de test

Niveau	Exemples	À vérifier
Fonction pure	file-selector, label-policy, parser, budget	Bornes, ordre, casse, erreurs et stabilité
Adaptateur	Groq, Gemini, OpenAI compatible	URL, en-têtes, payload, timeout et erreurs
Orchestration	PR, commentaire, Check Run	Appels Octokit, dégradation et idempotence
Persistance/UI	service, chiffrement, settings-page	Secret masqué, CSRF, changement de fournisseur

Les tests d'un handler doivent fournir un Octokit factice et vérifier les appels observables, sans reproduire son implémentation. Pour un fournisseur, remplacer fetch, contrôler le corps JSON et couvrir au minimum succès, 401/403, 404, 429, 5xx, réponse vide et JSON invalide.

12.3 Évaluation de la sélection

Scripts de diagnostic et d'évaluation

```
npm run evaluate:selection -- https://github.com/owner/repo/pull/123
npm run eval:selector -- evaluation/prs-annotées.json \
  --output evaluation/resultats.json
```

evaluate-file-selection inspecte une PR publique sans appeler le LLM. eval-selector compare les prédictions à un jeu annoté. selector-ablation permet de mesurer le titre seul, les rôles, les diffs classés et les diffs aléatoires. Une modification de pondération doit être évaluée sur le jeu complet, pas seulement sur une PR favorable.

12.4 Diagnostic en production

Symptôme	Vérification
Aucun commentaire	Webhook reçu, permission issues:write, exception dans readPullRequestData.
Aucune suggestion	Catalogue de labels, fournisseur résolu, seuil 0,70, message d'échec.
Contexte trop grand	contextLimitTokens, nonPatchEstimatedTokens, allocations et troncatures.
Bouton sans effet	Signature, headSha, PR liée à la Check Run et commentaire officiel.
/settings en 503	DATABASE_URL, OAuth et CONFIG_ENCRYPTION_KEY disponibles au démarrage.

13. Extension, déploiement et passation

13.1 Flux de modification recommandé

- Identifier l'invariant concerné et lire le module, ses tests et la documentation associée.
- Ajouter d'abord un test qui décrit le comportement attendu ou la régression observée.
- Modifier la plus petite couche responsable; éviter de déplacer une règle dans un handler.
- Exécuter build, lint, format:check, tests et, si nécessaire, l'évaluation de sélection.
- Mettre à jour README.md, docs et ce guide lorsque l'interface, une variable ou un flux change.

13.2 Recettes d'extension

Évolution	Fichiers principaux	Preuves attendues
Nouveau rôle ou score	file-selector.ts, constants.ts	Tests de précédence, score, diversité et contenu
Nouveau fournisseur	provider-configuration, adapter, factory, budget	Tests HTTP, erreurs, connexion et fenêtre
Nouvel événement	index.ts, handler, app.yml	Permissions, idempotence et payload fixture
Nouvelle donnée	migration, repository, service	Migration rejouable, transaction et audit
Nouvelle action GitHub	check-run.ts, handler, comment-state	Signature, SHA périmé et répétition

13.3 Déploiement

- Fournir PostgreSQL persistant, exécuter npm run db:migrate et sauvegarder la base.
- Injecter APP_ID, PRIVATE_KEY, WEBHOOK_SECRET, OAuth, DATABASE_URL et CONFIG_ENCRYPTION_KEY par un gestionnaire de secrets.
- Servir PUBLIC_BASE_URL en HTTPS; mettre à jour Setup URL, Callback URL et webhook de la GitHub App.
- Vérifier les quatre permissions et les quatre familles d'événements du manifeste.
- Exécuter une PR de fumée : commentaire, suggestion, Check Run, case cochée et nouveau commit.

- Surveiller taux d'erreur fournisseur, 429, latence webhook, jetons et erreur d'estimation.

13.4 Dettes techniques à connaître

Le système ne possède pas de queue, de verrou distribué ni de politique de concurrence entre deux analyses proches. Les singletons runtime sont initialisés paresseusement à partir de l'environnement. La rotation de `CONFIG_ENCRYPTION_KEY` n'est pas automatisée. Les fenêtres de modèles sont codées par familles et doivent être révisées. Enfin, les migrations sont manuelles et aucun pipeline CI n'est fourni dans le dépôt.